
Step 2:

Select the option :1 and key expires :0

```
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
  0 = key does not expire
  <n> = key expires in n days
  <n>w = key expires in n weeks
  <n>m = key expires in n months
  <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y
GnuPG needs to construct a user ID to identify your key.
```

Step 3:

Enter your name, Email ID , comment and create password

```
Real name: Arya Girishkumar
Email address: aryagirish101@gmail.com
Comment: heyyy!
You selected this USER-ID:
"Arya Girishkumar (heyyy!) <aryagirish101@gmail.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? O
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/kali/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/23CC5AE359AFE1637A83FA0BEE92804E5FD50159.rev'
public and secret key created and signed.

pub  rsa4096 2025-06-12 [SC]
     23CC5AE359AFE1637A83FA0BEE92804E5FD50159
uid  Arya Girishkumar (heyyy!) <aryagirish101@gmail.com>
sub  rsa4096 2025-06-12 [E]
```

Step 4:

Check the public keys

```
(kali㉿kali)-[~]
└─$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
/home/kali/.gnupg/pubring.kbx

pub  rsa4096 2025-06-12 [SC]
     23CC5AE359AFE1637A83FA0BEE92804E5FD50159
uid  [ultimate] Arya Girishkumar (heyyy!) <aryagirish101@gmail.com>
sub  rsa4096 2025-06-12 [E]
```

Step 5:

check the private keys

```
(kali㉿kali)-[~]
└─$ gpg --list-secret-keys
/home/kali/.gnupg/pubring.kbx
-----
sec  rsa4096 2025-06-12 [SC]
      23CC5AE359AFE1637A83FA0BEE92804E5FD50159
uid  following [ultimate] Arya Girishkumar (heyyy!) <aryagirish101@gmail.com>
ssb  rsa4096 2025-06-12 [E]
```

Step 6:

Create a file and export the public key

```
(kali㉿kali)-[~]
└─$ gpg --armor --export aryagirish101@gmail.com > Arya_Girishkumar_public_key.asc
```

Step 7:

Create a file and enter a message

```
(kali㉿kali)-[~]
└─$ echo "don't open this message" > secret.txt
```

Step 8:

Encrypt the file

```
(kali㉿kali)-[~]
└─$ gpg --encrypt --recipient aryagirish101@gmail.com secret.txt
```

Step 9:

Decrypt the file

```
(kali㉿kali)-[~]
└─$ gpg --output decrypted.txt --decrypt secret.txt.gpg
gpg: encrypted with 4096-bit RSA key, ID 1F2594B82E66E7B2, created 2025-06-12
      "Arya Girishkumar (heyyy!) <aryagirish101@gmail.com>"
```

Step 10:

Create a digital sign

```
(kali㉿kali)-[~]
└─$ gpg --armor --sign secret.txt
```

Step 11:

Verify the secret file

```
(kali㉿kali)-[~]
└─$ gpg --verify secret.txt.asc
gpg: Signature made Thu 12 Jun 2025 01:15:09 AM EDT
gpg: using RSA key 23CC5AE359AFE1637A83FA0BEE92804E5FD50159
gpg: Good signature from "Arya Girishkumar (heyyy!) <aryagirish101@gmail.com>" [ultimate]
gpg: WARNING: not a detached signature; file 'secret.txt' was NOT verified!
```